

Evidencias de la actividad práctica de la sesión 5

Clean Architecture y MVVM en el módulo de productos

Estudiante	Roberto Valencia
Proyecto	PharmaMobil
Sesión	Sesión 5
Rama de trabajo	feature/clean-mvvm

Resultado general

El módulo de productos quedó reorganizado por responsabilidades y conserva su funcionamiento. La pantalla representa correctamente los estados de carga, vacío, contenido y error, mientras que la lógica se mantiene en el código compartido.

Objetivo de la implementación

El trabajo consistió en reorganizar el módulo de productos aplicando Clean Architecture junto con el patrón MVVM. El propósito fue separar la interfaz, la lógica de negocio y el acceso a datos para evitar duplicaciones y facilitar que, más adelante, un backend REST pueda reemplazar al repositorio en memoria.

Distribución de responsabilidades

Capa	Elementos principales	Responsabilidad
Domain	Modelo, repository y use case	Define las reglas del negocio sin depender de la interfaz ni de la fuente de datos.
Data	Repositorio en memoria	Implementa el contrato del dominio y simula el acceso a información.
Presentation	Screen, ViewModel y UI state	Recibe eventos, conserva el estado y lo representa mediante Compose.
DI	AppModule de Koin	Construye y conecta las dependencias sin crearlas directamente en la pantalla.

La interfaz depende del ViewModel; el ViewModel utiliza los casos de uso; y estos trabajan con el contrato ProductoRepository. Por eso, cambiar la implementación del repositorio no obliga a modificar la pantalla.

Evidencias obtenidas

Las siguientes capturas muestran el comportamiento solicitado en Android, la organización final del proyecto y la ejecución satisfactoria de las pruebas automatizadas.

Evidencia 1 Estado de carga

La pantalla muestra un indicador de progreso mientras el ViewModel espera la respuesta del repositorio. Durante este estado se evita que el usuario vuelva a registrar información.

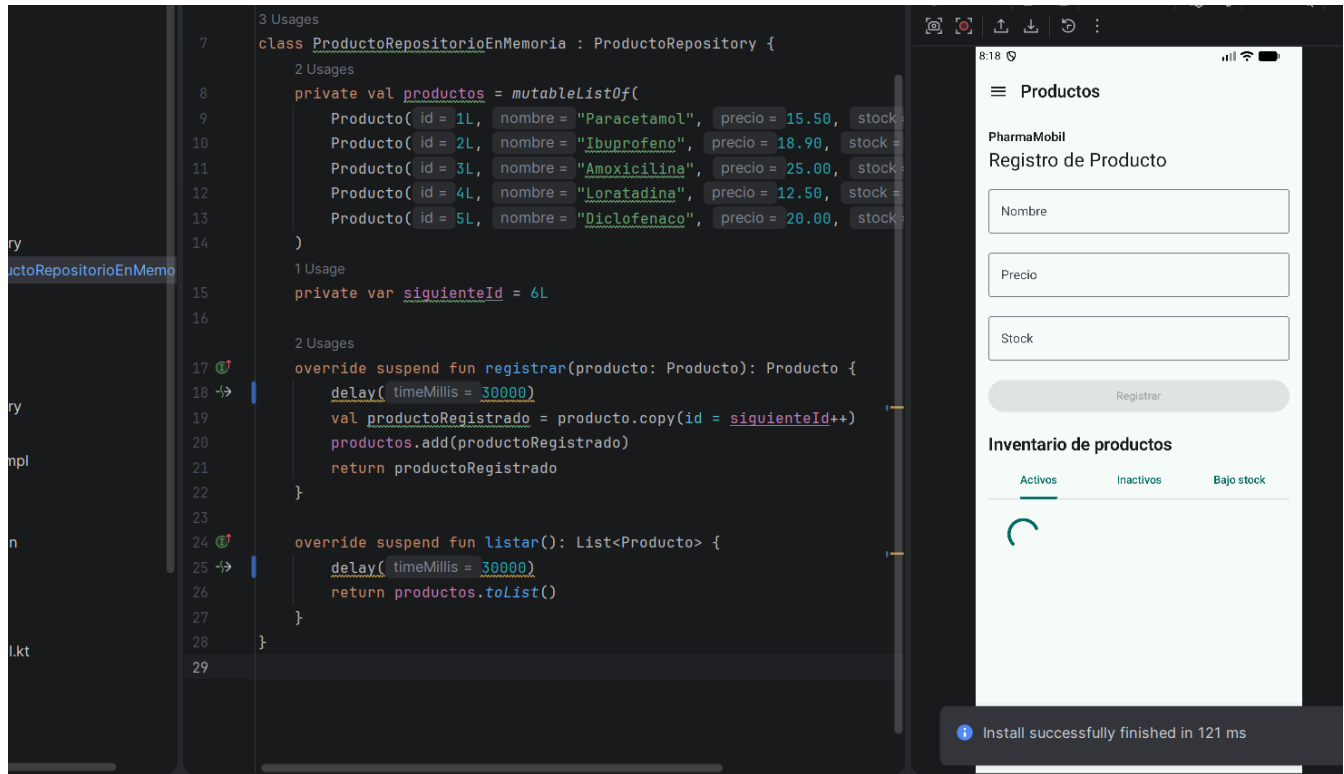


Figura 1. Estado de carga.

Evidencia 2 Estado vacío

Cuando el repositorio no contiene elementos, la interfaz informa de forma clara que todavía no existen productos registrados.

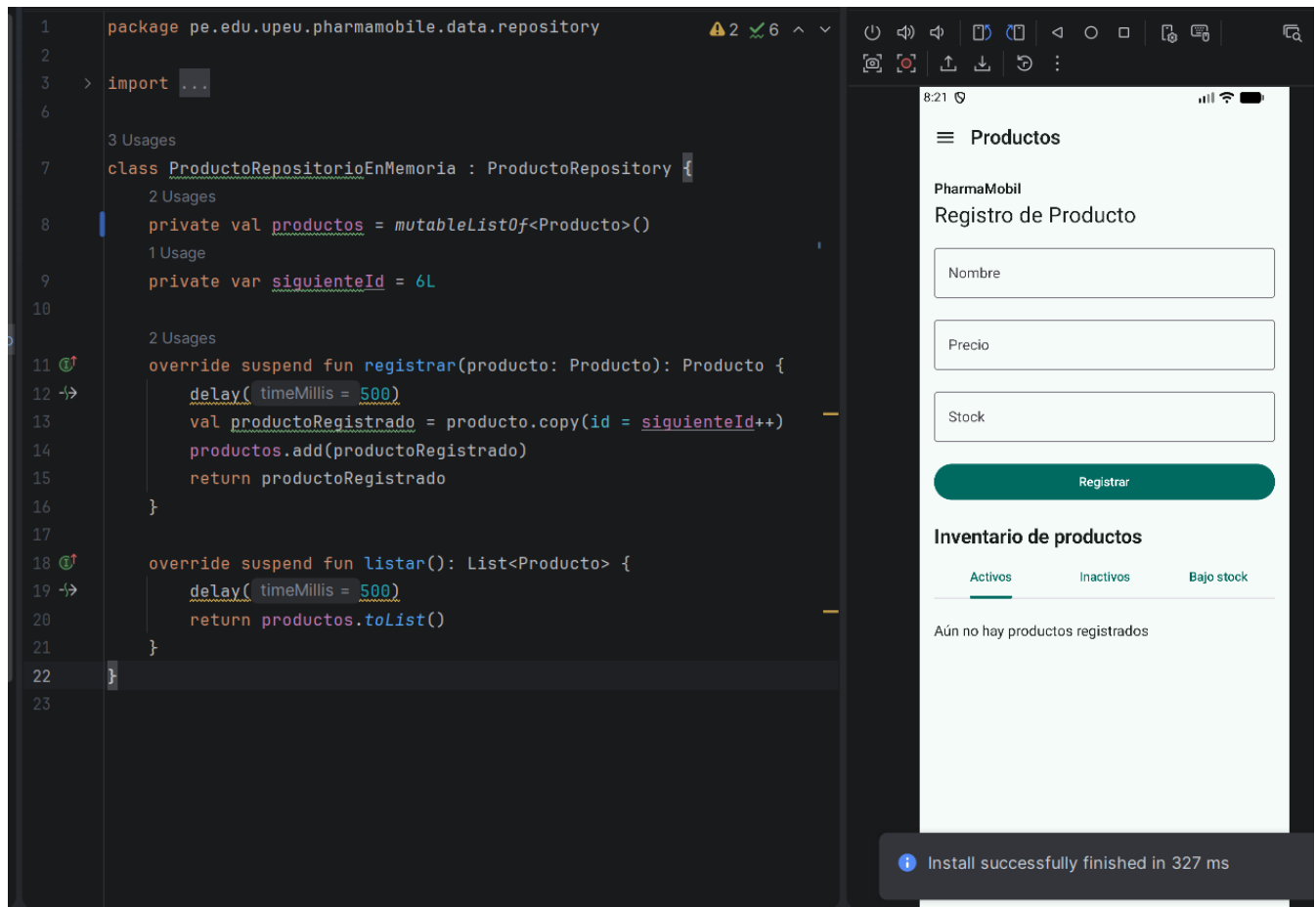


Figura 2. Estado vacío.

Evidencia 3 Estado con productos

La información entregada por el caso de uso se presenta en tarjetas. Se observan nombre, precio, stock y estado de cada producto.

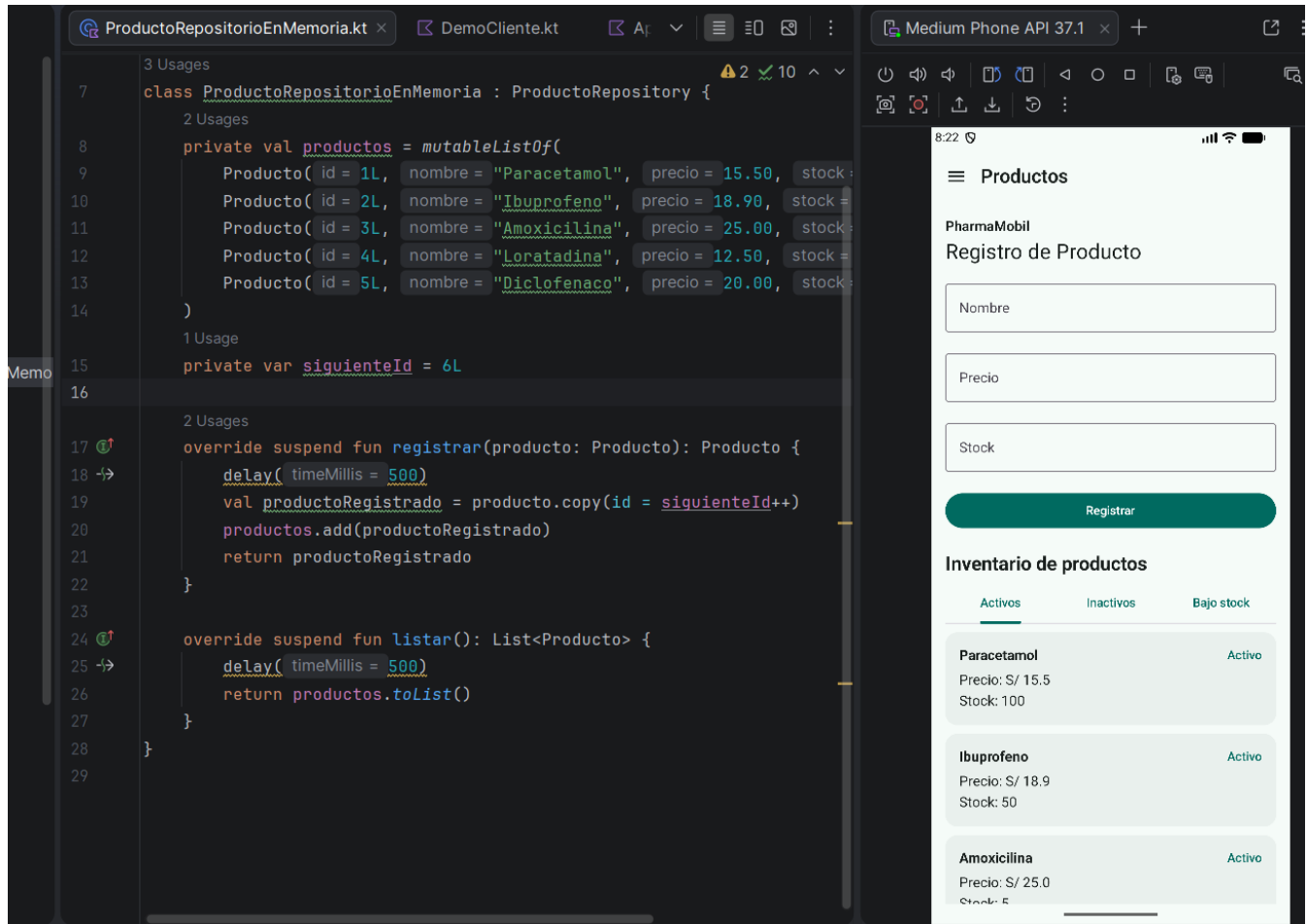


Figura 3. Estado con productos.

Evidencia 4 Estado de error

La excepción simulada en el repositorio llega al ViewModel y se representa mediante un mensaje comprensible y la opción Reintentar.

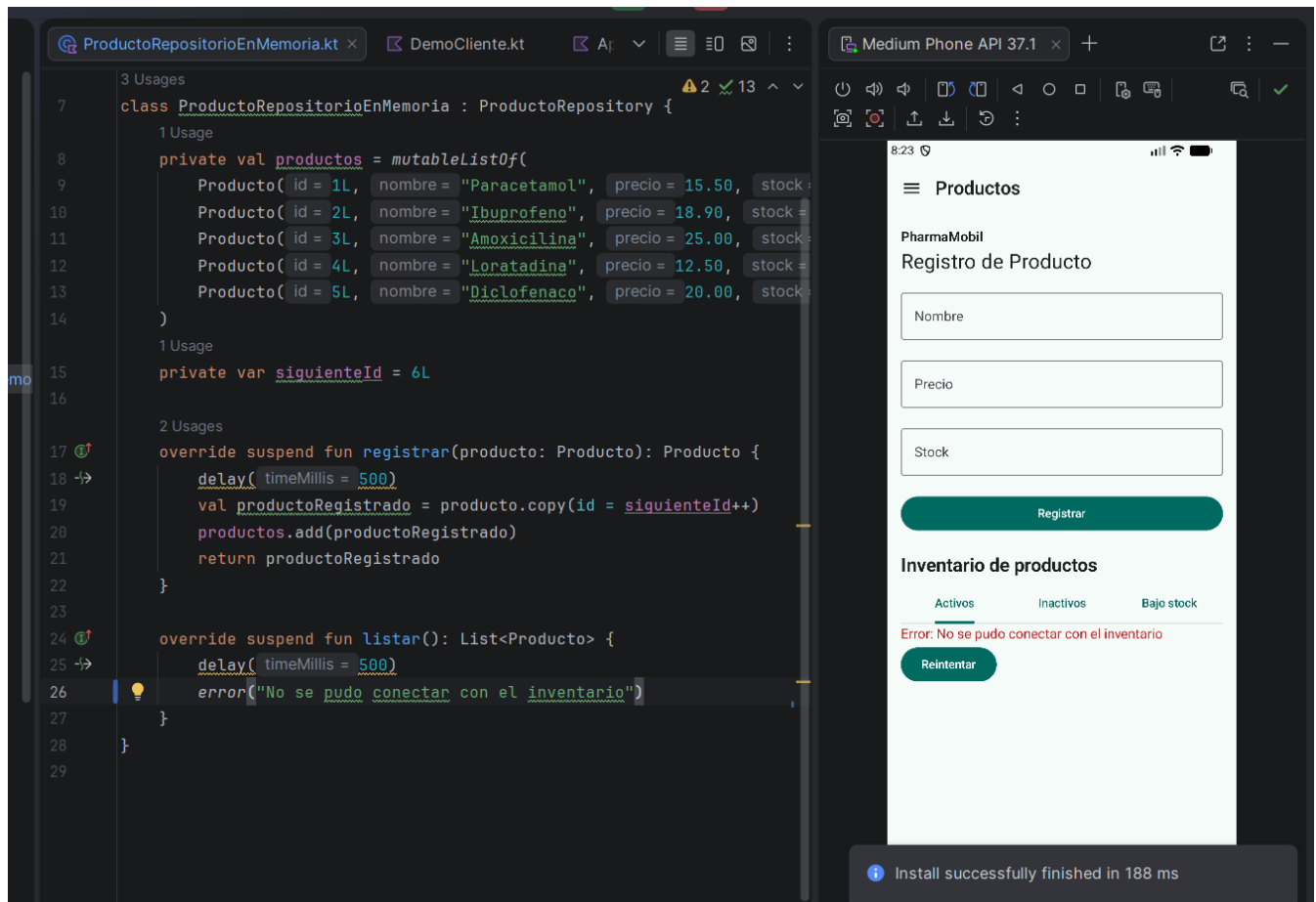


Figura 4. Estado de error.

Evidencia 5 Organización Clean Architecture y MVVM

El módulo quedó separado en data, domain, di y presentation. Esta distribución permite reemplazar el repositorio en memoria por una API REST sin reescribir la pantalla.

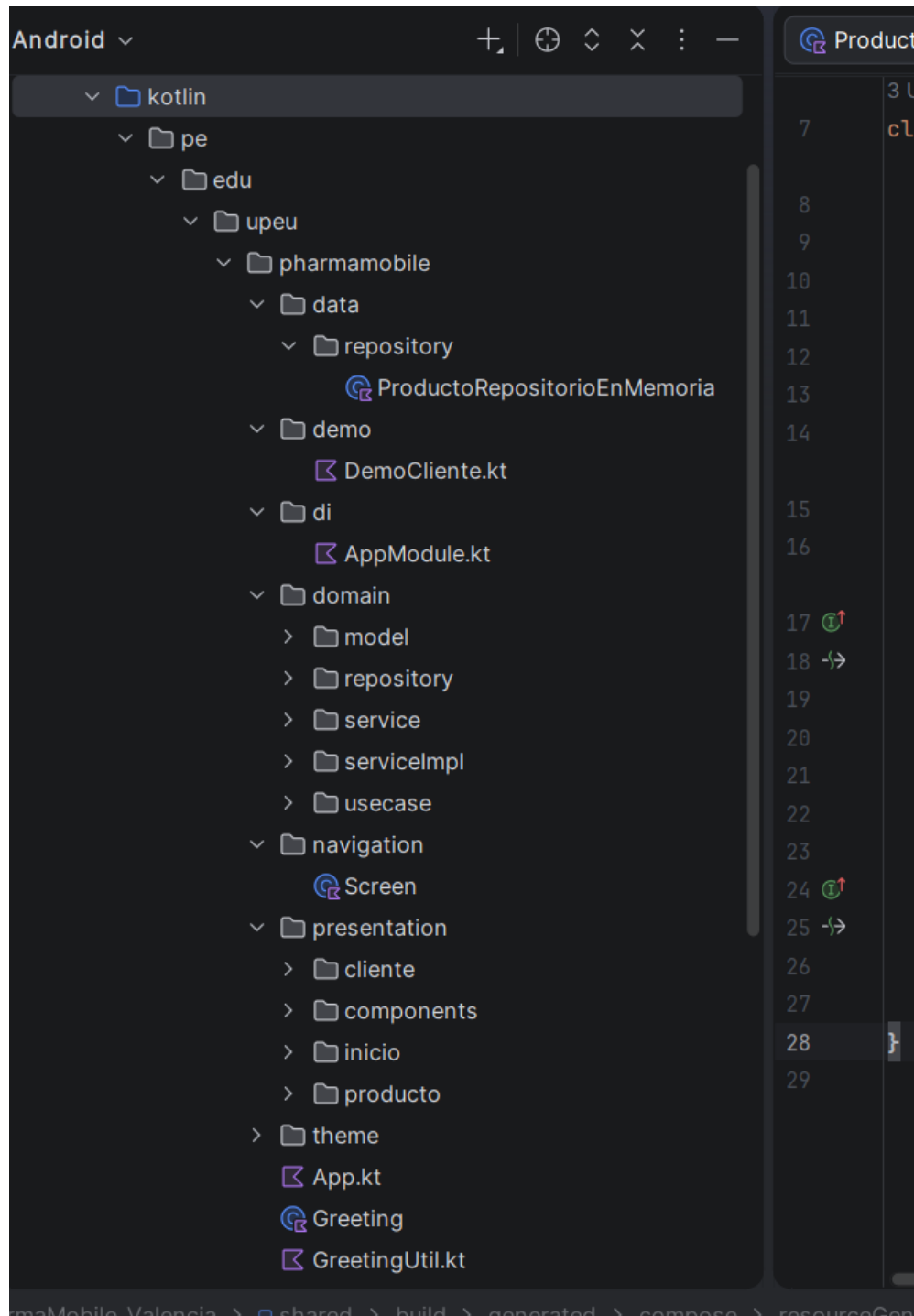


Figura 5. Organización Clean Architecture y MVVM.

Evidencia 6 Ejecución de pruebas

Las pruebas del módulo compartido finalizaron correctamente. El resultado BUILD SUCCESSFUL confirma que la lógica común compila y supera las verificaciones en Android.

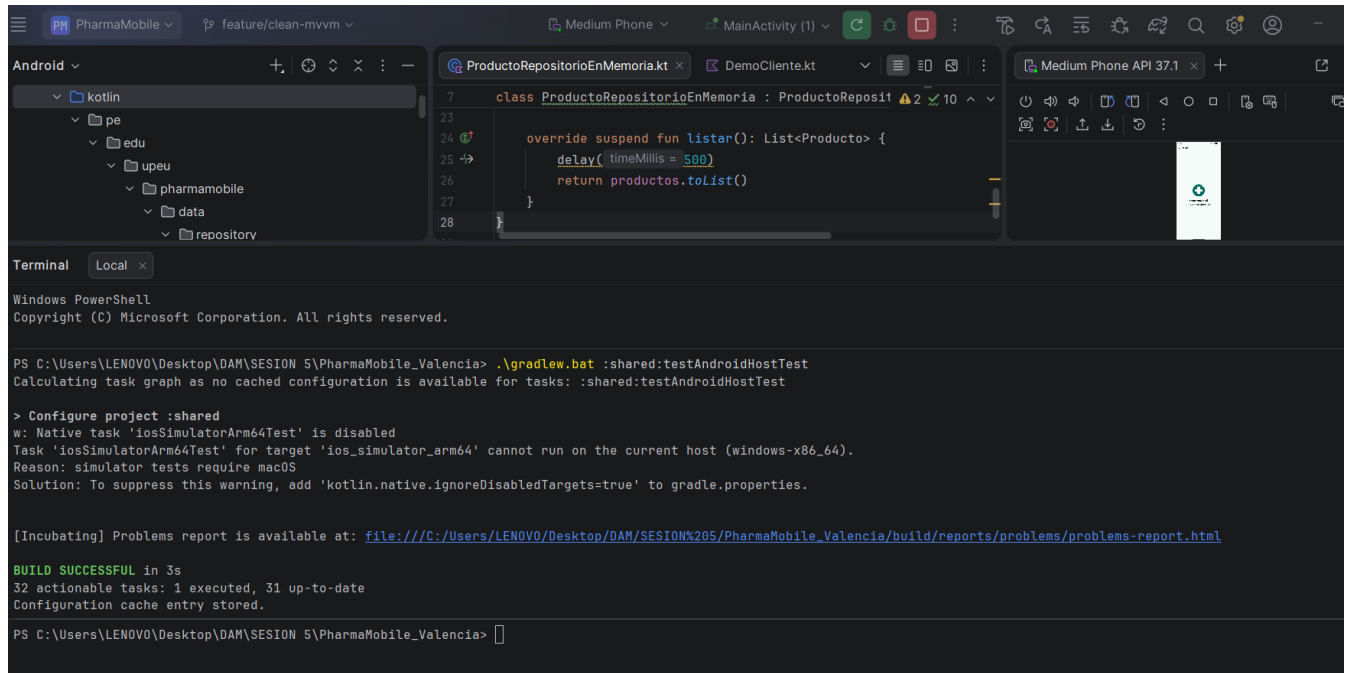


Figura 6. Ejecución de pruebas.

Conclusiones

- La separación en capas permite que cada archivo tenga una responsabilidad clara y reduce el acoplamiento entre la interfaz y los datos.
- MVVM concentra el estado y las acciones del usuario en el ViewModel, por lo que la pantalla se limita a mostrar información y enviar eventos.
- Los cuatro estados de la interfaz fueron comprobados en Android: carga, lista vacía, lista con productos y error recuperable.
- Las pruebas del código compartido terminaron con BUILD SUCCESSFUL, confirmando que la reorganización mantiene el proyecto compilable y funcional.

Con esta estructura, el siguiente paso puede ser implementar un repositorio remoto conectado a una API REST y seleccionarlo desde Koin, sin cambiar la pantalla de productos.